

Projet Final : Classification d'Images par Apprentissage Automatique Traditionnel et Réseaux de Neurones Convolutifs (CNN)

1. Énoncé

Objectif du projet:

Développer une application complète de classification d'images visant à:

- Comparer l'efficacité de l'apprentissage automatique traditionnel (SVM, k-NN, Arbre de Décision, Naïve Bayes) basé sur des caractéristiques extraites avec des modèles pré-entraînés (VGG16, AlexNet, InceptionV3) ;
- Comparer ces résultats avec ceux obtenus en entraînant directement un modèle CNN personnalisé sur les images brutes;
- Analyser les performances à l'aide de critères quantitatifs (accuracy, precision, F1-score, etc.) et qualitatifs (temps d'entraînement, difficulté d'implémentation).

En d'autres termes, concevoir et comparer deux pipelines de classification d'images:

- Pipeline 1:
 - Extraction de caractéristiques à partir de modèles pré-entraînés (VGG16, AlexNet, InceptionV3).
 - Entraînement de modèles traditionnels : SVM, k-NN, Arbre de Décision, Naïve Bayes.
- Pipeline 2:
 - Entraînement direct d'un modèle CNN personnalisé à partir des images brutes, sans extraction intermédiaire.

2. Introduction

La classification d'images est un problème central en vision par ordinateur. Deux grandes approches coexistent:

- L'apprentissage automatique traditionnel: nécessite une étape d'extraction de caractéristiques.
- L'apprentissage profond / réseau de neurones convolutifs (CNN): apprend directement des images, mais avec un coût en ressources plus important.

Ce projet vous permettra de comparer concrètement ces deux méthodes et de mieux comprendre leurs forces et limites.

3. Problématique

- Quels sont les avantages et inconvénients des modèles traditionnels vs. CNN?
- Quelle approche est plus adaptée en fonction du type de données et du contexte d'utilisation (temps de calcul, ressources, facilité de mise en œuvre)?

4. Développement

4.1 Architecture globale du système

PHASE 1 : Pipeline Traditionnel avec Extraction de Caractéristiques

1. Préparation des données:

- Redimensionnement et normalisation des images.
- Séparation en:
 - 70% entraînement et 30% test.

2. Extraction de caractéristiques:

- Utilisation de VGG16, AlexNet et InceptionV3 (pré-entraînés sur ImageNet).
- Extraction des vecteurs avant la couche fully connected finale.
- Sauvegarde des résultats sous une des formes ci-dessous:
 - CSV :
| feature_1 | feature_2 | ... | label |
 - Fichiers NumPy (.npy).

3. Entraînement des modèles classiques :

- SVM (kernel à choisir et justifier).
- k-NN (k à optimiser).
- Arbre de Décision.
- Naïve Bayes.

4. Évaluation des modèles :

- Metrics : Accuracy, Precision, Recall, F1-score.
- Temps d'entraînement.
- Matrice de confusion et courbe ROC-AUC (si applicable).

PHASE 2 : Pipeline CNN

1. Conception d'un CNN personnalisé

- Architecture minimale :
 - 3 à 5 couches convolutionnelles.
 - Pooling, Dropout, Batch Normalisation.
 - Fully connected layers.
- Justification de l'architecture choisie dans le rapport.

2. Entraînement du CNN

- Optimisation des hyperparamètres (epochs, learning rate, batch size, etc.).
- Utilisation de GPU (si possible).

3. Évaluation du CNN

- Metrics identiques à celles des modèles traditionnels.
- Comparaison directe avec les résultats obtenus dans la Phase 1.

PHASE 3 : Analyse Comparative

- Tableaux comparatifs :
 - Performance (accuracy, F1-score, etc.).
 - Temps d'entraînement.
 - Taille des modèles (nombre de paramètres).
- Analyse critique et recommandations d'usage.

4.2 Technologies et outils suggérés

- Python 3.x
- Tensorflow
- Pytorch
- Scikit-learn
- Pandas, NumPy
- Matplotlib, Seaborn

5. Jeux de données

- DTD
- Iris Flowers
- COVID-19 X-Ray
- Wildfire Satellite Images

Les jeux de données seront fournis sur Moodle ou Teams.

6. Livrable

- Rapport PDF complet.
- Dossier compressé contenant :
 - Scripts Python.
 - Jupyter Notebook
 - Modèles sauvegardés.
 - Fichiers CSV ou .npy.
 - README.txt et requirements.txt.
- Vidéo de démonstration obligatoire (2–3 minutes).

7. Barème de correction détaillé

Critère	Pondération	Description
Architecture et Conception	40%	- Clarté et logique de l'organisation des deux pipelines. - Respect du découpage : prétraitement, extraction, entraînement, évaluation. - Gestion structurée des jeux de données, des fichiers CSV et modèles. - Qualité de l'organisation des fichiers et scripts (bonne pratique : modularisation, nommage clair).
Fonctionnalités et Code	35%	- Fonctionnalité effective : extraction, entraînement, évaluation. - Robustesse du code (gestion des erreurs, modularisation). - Respect des bonnes pratiques Python.
Analyse comparative et Visualisation	15%	- Présence de graphiques et tableaux pertinents. - Qualité de l'analyse critique entre les deux approches. - Clarté et lisibilité des résultats.
Documentation et Rapport	10%	- Présentation complète : introduction, problématique, méthodologie, résultats, discussion, conclusion.
Total	100%	